

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - CƠ - TIN HỌC

—o0o—



TỐI ƯU RỜI RẠC
Discrete Optimization

Mã lớp học phần: **MAT3323**

Sinh viên: **LƯU VĂN VIỆT**

Lớp: **K65A2 TOÁN - TIN**

Hà Nội, tháng 1 năm 2024

Mục lục

§1. Đồ thị và duyệt đồ thị	4
1 Duyệt theo chiều rộng và duyệt theo chiều sâu	4
2 Tính liên thông và chu trình của đồ thị	6
3 Đồ thị có hướng phi chu trình	7
§2. Cây bao trùm	10
1 Thuật toán Prim	10
2 Thuật toán Kruskal	12
§3. Bài toán đường đi ngắn nhất	13
1 Thuật toán Dijkstra	13
2 Thuật toán Bellman-Ford	13
§4. Bài toán luồng cực đại, lát cắt cực tiểu	15
1 Lát cắt và luồng trên mạng	15
2 Thuật toán Ford-Fulkerson	16
§5. Bài toán cặp ghép	17
1 Bài toán ghép cặp	17
2 Thuật toán Hungarian	18
§6. Bài toán ghép cặp bền vững	20
1 Bài toán ghép cặp bền vững	20
2 Thuật toán Gale-Shaley	20
§7. Các mô hình tối ưu nguyên	22
1 Mô hình IP cho bài toán đường đi ngắn nhất với trọng số cạnh dương.	22
2 Mô hình LP cho bài toán luồng cực đại.	22

3	Mô hình IP cho bài toán cây bao trùm nhỏ nhất.	23
4	Mô hình IP cho bài toán ghép cặp hoàn hảo với trọng số cực tiểu.	23
5	Mô hình IP cho bài toán phân công	24
6	Mô hình IP cho bài toán TSP	24
7	Mô hình IP cho bài toán Knapsack	25

§1. Đồ thị và duyệt đồ thị

1 Duyệt theo chiều rộng và duyệt theo chiều sâu

Duyệt theo chiều rộng

Algorithm 1 Duyệt theo chiều rộng

Input: Đồ thị G và đỉnh xuất phát v_0 .

Output: Danh sách thứ tự các đỉnh khi duyệt theo chiều rộng.

PROCEDURE BFS(G, v_0):

```

1: # Tạo một mảng lưu trữ trạng thái duyệt các đỉnh của đồ thị,  $state(v) = 1$  nghĩa là đỉnh  $v$  đã được
   duyệt,  $state(v) = 0$  nghĩa là đỉnh  $v$  chưa được duyệt
2:  $state \leftarrow \emptyset$ 
3: # Tạo một hàng đợi ưu tiên lưu thứ tự duyệt các đỉnh
4:  $Q \leftarrow \emptyset$ 
5: # Tạo một mảng lưu kết quả khi duyệt các đỉnh theo thuật toán BFS
6:  $L \leftarrow \emptyset$ 
7: # Đánh dấu các đỉnh ngoài đỉnh gốc là chưa được duyệt
8: for  $v \in V_G \setminus \{v_0\}$  do
9:    $state(v) \leftarrow 0$ 
10: end for
11: # Đánh dấu đỉnh gốc đã được duyệt và thêm vào hàng đợi ưu tiên
12:  $state(v_0) \leftarrow 1$ 
13:  $enqueue(Q, v_0)$ 
14: while  $Q$  is not empty do
15:   # Xóa đỉnh đầu hàng đợi và thêm vào mảng kết quả sau đó kiểm tra xem các đỉnh kề đỉnh đó đã
   được duyệt hay chưa, nếu chưa thì thêm đỉnh đó vào hàng đợi.
16:    $u \leftarrow dequeue(Q)$ 
17:    $L.add(u)$ 
18:   for  $v \in G.Adj(u)$  do
19:     if  $state(v) = 0$  then
20:        $state(v) \leftarrow 1$ 
21:        $enqueue(Q, v)$ 
22:     end if
23:   end for
24: end while

```

END PROCEDURE

Đánh giá độ phức tạp của BFS: Thuật toán BFS duyệt qua n đỉnh, tại mỗi đỉnh duyệt qua các đỉnh kề với đỉnh đó. Với n đỉnh thì số lần duyệt danh sách kề các đỉnh là $2m$ lần. Do đó tổng độ phức tạp của thuật toán là $O(m + n)$.

Duyệt theo chiều sâu

Algorithm 2 Duyệt theo chiều sâu

Input: Đồ thị G .**Output:** Danh sách thứ tự các đỉnh khi duyệt theo chiều sâu.**PROCEDURE** DFS(G):

```

1: # Tạo một mảng lưu trữ trạng thái duyệt các đỉnh của đồ thị,  $state(v) = 1$  nghĩa là đỉnh  $v$  đã được
   duyệt,  $state(v) = 0$  nghĩa là đỉnh  $v$  chưa được duyệt
2:  $state \leftarrow \emptyset$ 
3: # Tạo một mảng lưu kết quả khi duyệt các đỉnh theo thuật toán DFS
4:  $L \leftarrow \emptyset$ 
5: # Đánh dấu các đỉnh trong đồ thị chưa được duyệt
6: for  $v \in V_G$  do
7:    $state(v) \leftarrow 0$ 
8: end for
9: # Duyệt qua các đỉnh và thực hiện tìm kiếm theo chiều sâu tại các đỉnh chưa được đánh dấu là đã
   duyệt.
10: for  $v \in V_G$  do
11:   if  $state(v) = 0$  then
12:     DFS_root( $G, v, L, state$ )
13:   end if
14: end for

```

END PROCEDURE**PROCEDURE** DFS_root($G, v_0, L, state$):

```

1:  $L.add(v_0)$ 
2:  $state(v_0) \leftarrow 1$ 
3: for  $v \in G.Adj(v_0)$  do
4:   if  $state(v) = 0$  then
5:     DFS_root( $G, v, state$ )
6:   end if
7: end for

```

END PROCEDURE

Đánh giá độ phức tạp của DFS: Thuật toán DFS duyệt qua n đỉnh, tại mỗi đỉnh duyệt qua các đỉnh kề với đỉnh đó. Với n đỉnh thì số lần duyệt danh sách kề các đỉnh là $2m$ lần. Do đó tổng độ phức tạp của thuật toán là $O(m + n)$.

2 Tính liên thông và chu trình của đồ thị

Kiểm tra xem 1 đồ thị có liên thông hay không

Algorithm 3 Kiểm tra tính liên thông của đồ thị

Input: Đồ thị G .

Output: Nếu đồ thị G liên thông thì trả về **True** ngược lại trả về **False**.

PROCEDURE check_if_connected(G):

```

1: # Tạo một mảng lưu trữ trạng thái các đỉnh của đồ thị khi duyệt qua DFS,  $state(v) = 1$  nghĩa là
   đỉnh  $v$  đã được duyệt,  $state(v) = 0$  nghĩa là đỉnh  $v$  chưa được duyệt
2:  $state \leftarrow \emptyset$ 
3: # Đánh dấu các đỉnh trong đồ thị chưa được duyệt
4: for  $v \in V_G$  do
5:    $state(v) \leftarrow 0$ 
6: end for
7: # Bắt đầu từ một đỉnh bất kỳ, thực hiện tìm kiếm theo chiều sâu từ đỉnh đó.
8:  $v_0 \leftarrow \text{any node in } G$ 
9:  $DFS\_root(G, v_0, state)$ 
10: # Nếu trạng thái của các đỉnh sau khi thực hiện DFS với 1 đỉnh bất kỳ đều bằng 1 thì đồ thị liên
    thông, ngược lại đồ thị không liên thông.
11: for  $v \in V_G$  do
12:   if  $state(v) = 0$  then
13:     return False
14:   end if
15: end for
16: return True

```

END PROCEDURE

PROCEDURE DFS_root($G, v_0, state$):

```

1:  $state(v_0) \leftarrow 1$ 
2: for  $v \in G.Adj(v_0)$  do
3:   if  $state(v) = 0$  then
4:      $DFS\_root(G, v, state)$ 
5:   end if
6: end for

```

END PROCEDURE

Đánh giá độ phức tạp của thuật toán kiểm tra tính liên thông: Độ phức tạp của thuật toán này là $O(m + n)$.

Kiểm tra xem 1 đồ thị có chu trình hay không

Algorithm 4 Kiểm tra xem đồ thị có chu trình hay không

Input: Đồ thị G .

Output: Nếu đồ thị G có chu trình thì trả về **True** ngược lại trả về **False**.

PROCEDURE detect_cycle(G):

```

1:  state  $\leftarrow \emptyset$ 
2:  rec_state  $\leftarrow \emptyset$ 
3:  for  $v \in V_G$  do
4:      state( $v$ )  $\leftarrow 0$ 
5:      rec_state( $v$ )  $\leftarrow 0$ 
6:  end for
7:  for  $v \in V_G$  do
8:      if DFS_root( $G, v, state, rec\_state$ ) = True then
9:          return True
10:     end if
11: end for
12: return False

```

END PROCEDURE

PROCEDURE DFS_root($G, v_0, state, rec_state$):

```

1:  if rec_state( $v_0$ ) = 1 then
2:      return True
3:  end if
4:  if state( $v_0$ ) = 1 then
5:      return False
6:  end if
7:  state( $v_0$ )  $\leftarrow 1$ 
8:  rec_state( $v_0$ )  $\leftarrow 1$ 
9:  for  $v \in G.Adj(v_0)$  do
10:     if DFS_root( $G, v, state, rec\_state$ ) = True then
11:         return True
12:     end if
13: end for
14: rec_state( $v_0$ )  $\leftarrow 0$ 
15: return False

```

END PROCEDURE

Đánh giá độ phức tạp của thuật toán kiểm tra chu trình: Độ phức tạp của thuật toán này là $O(m + n)$.

3 Đồ thị có hướng phi chu trình

Thuật toán tìm thứ tự Topo trong DAG

Algorithm 5 Tìm thứ tự Topo của một DAG

Input: 1 Đồ thị có hướng phi chu trình G .

Output: 1 thứ tự Topo trên DAG Input.

PROCEDURE find_topological_order(G):

```

1: # Tạo một mảng lưu thứ tự Topo
2:  $T \leftarrow \emptyset$ 
3: # Tạo một mảng chứa các đỉnh không có cạnh đi vào của đồ thị  $G$ 
4:  $S \leftarrow \emptyset$ 
5: # Với  $V_G$  là ký hiệu tập đỉnh của  $G$ ,  $\deg^-(v)$  là ký hiệu bậc vào của đỉnh  $v$ 
6: for  $v \in V_G$  do
7:   if  $\deg^-(v) = 0$  then
8:      $S \leftarrow S \cup \{v\}$ 
9:   end if
10: end for
11: while  $|S| < |V_G|$  do
12:   # Lấy 1 đỉnh  $s$  không có cạnh đi vào của đồ thị  $G$  ở bước hiện tại từ  $S$  và thêm vào  $T$ 
13:    $S \leftarrow S \setminus \{s\}$ 
14:    $T \leftarrow T \cup \{s\}$ 
15:   for  $e = (s, u) \in E_G$  do
16:     # Giảm bậc vào của các đỉnh nhận cạnh  $e$  là cạnh đi vào
17:      $\deg^-(u) \leftarrow \deg^-(u) - 1$ 
18:     # Nếu bậc vào của đỉnh  $u$  bằng 0 tức là đỉnh  $u$  không còn cạnh đi vào thì ta thêm  $u$  vào  $S$ 
19:     if  $\deg^-(u) = 0$  then
20:        $S \leftarrow S \cup \{u\}$ 
21:     end if
22:   end for
23: end while

```

END PROCEDURE

Đánh giá thuật toán:

- Với thao tác khởi tạo mảng S chứa các đỉnh không có cạnh đi vào, ta cần duyệt qua n đỉnh của đồ thị ban đầu, mỗi lần duyệt cần xem xét bậc vào của đỉnh nên độ phức tạp của thao tác này là $O(|V|)$.
- Với vòng lặp **While** dùng để thêm các đỉnh vào mảng T , ta cần thêm tất cả các đỉnh của $\mathbf{G}$ vào T nên cần duyệt qua n đỉnh, tại mỗi đỉnh s ta duyệt các cạnh đi ra từ s . Do đó vòng lặp **While** duyệt tất cả các cạnh đi ra của các đỉnh trong $\mathbf{G}$ và có độ phức tạp là $O(|E|)$
- Vậy độ phức tạp của thuật toán tìm thứ tự Topo trên là $O(|V| + |E|)$

§2. Cây bao trùm

1 Thuật toán Prim

Thuật toán Prim:

Algorithm 6 Thuật toán Prim

Input: Đồ thị G .

Output: Cây bao trùm tối thiểu trên G .

PROCEDURE prim_algorithm(G):

```

1:  $T \leftarrow \emptyset$ 
2:  $S \leftarrow \emptyset$ 
3:  $s \leftarrow$  any node in  $V_G$ 
4: for  $v \in V_G$  and  $v \neq s$  do
5:    $\pi(v) \leftarrow \infty$ 
6:    $pred[v] = null$ 
7:    $\pi(s) = 0$ 
8: end for
9: # Tạo hàng đợi ưu tiên
10:  $pq \leftarrow priorityQueue()$ 
11: for  $v \in V_G$  do
12:    $insert(pq, v, \pi(v))$ 
13: end for
14: while  $pq$  is not empty do
15:    $u \leftarrow delete\_min(pq)$ 
16:    $S \leftarrow S \cup \{u\}$ 
17:    $T \leftarrow T \cup \{pred[u]\}$ 
18:   for  $e = (u, v) \in E_G$  and  $v \notin S$  do
19:     if  $c_e < \pi(v)$  then
20:        $decrease\_key(pq, v, c_e)$ 
21:        $\pi(v) \leftarrow c_e$ 
22:        $pred[v] = e$ 
23:     end if
24:   end for
25: end while

```

END PROCEDURE

Trong thuật toán, ta thực hiện các quá trình sau:

- Q_1 : Khởi tạo trọng số, duyệt qua n đỉnh và gán trọng số là ∞ hoặc 0 cho mỗi đỉnh.
- Q_2 : Duyệt qua n đỉnh và thêm vào hàng đợi ưu tiên pq .
- Q_3 : Lần lượt thêm các cạnh và đỉnh vào cây bằng vòng lặp **While**. Do cần

thêm n đỉnh vào cây nên vòng lặp **while** sẽ lặp n lần, mỗi lần lặp thực hiện việc tìm và xóa **min** khỏi pq , sau đó duyệt qua các cạnh liên thuộc với đỉnh được xóa khỏi pq và thực hiện cập nhật trọng số qua $decrease_key$. Do đó, ta thực hiện n lần $delete_min$ và m lần $decrease_key$.

1. Cài đặt pq bằng cấu trúc dữ liệu **Array**.

- Q_1 : Độ phức tạp của thao tác này là $O(n)$
- Q_2 : Thêm n phần tử vào pq với độ phức tạp là $O(1)$ cho mỗi lần thêm phần tử nên độ phức tạp của quá trình này là $O(n)$.
- Q_3 : Thực hiện n lần tìm và xóa **min** với độ phức tạp là $O(n)$ cho mỗi lần tìm và xóa **min**, m lần cập nhật trọng số với độ phức tạp là $O(1)$ cho việc cập nhật trọng số nên tổng độ phức tạp của quá trình này là $O(n^2) + O(m)$

Tổng độ phức tạp khi cài đặt thuật toán bằng cấu trúc dữ liệu **Array** là $O(n^2 + m) = O(n^2)$

2. Cài đặt pq bằng cấu trúc dữ liệu **Binary Heap**.

- Q_1 : Độ phức tạp của thao tác này là $O(n)$
- Q_2 : Thêm n phần tử vào pq với độ phức tạp là $O(\log(n))$ cho mỗi lần thêm phần tử nên độ phức tạp của quá trình này là $O(n\log(n))$.
- Q_3 : Thực hiện n lần tìm và xóa **min** với độ phức tạp là $O(\log(n))$ cho mỗi lần tìm và xóa **min**, m lần cập nhật trọng số với độ phức tạp là $O(\log(n))$ cho việc cập nhật trọng số nên tổng độ phức tạp của quá trình này là $O(m\log(n))$

Tổng độ phức tạp khi cài đặt thuật toán bằng cấu trúc dữ liệu **Binary Heap** là

$$O(n\log(n) + m\log(n)) = O(m\log(n))$$

3. Cài đặt pq bằng cấu trúc dữ liệu **Fibonacci Heap**.

- Q_1 : Độ phức tạp của thao tác này là $O(n)$

- Q_2 : Thêm n phần tử vào pq với độ phức tạp là $O(1)$ cho mỗi lần thêm phần tử nên độ phức tạp của quá trình này là $O(n)$.
- Q_3 : Thực hiện n lần tìm và xóa **min** với độ phức tạp là $O(\log(n))$ cho mỗi lần tìm và xóa **min**, m lần cập nhật trọng số với độ phức tạp là $O(1)$ cho việc cập nhật trọng số nên tổng độ phức tạp của quá trình này là $O(n\log(n) + m)$

Tổng độ phức tạp khi cài đặt thuật toán bằng cấu trúc dữ liệu **Fibonacci Heap** là

$$O(n\log(n) + m)$$

2 Thuật toán Kruskal

Thuật toán Kruskal:

Algorithm 7 Thuật toán Kruskal

Input: Đồ thị G .

Output: Cây bao trùm tối thiểu trên G .

PROCEDURE kruskal_algorithm(G):

```

1: # Sắp xếp  $m$  cạnh theo độ lớn của trọng số tăng dần
2:  $sort\_by\_weights(E_G)$ 
3:  $T \leftarrow \emptyset$ 
4: for  $v \in V_G$  do
5:    $make\_set(v)$ 
6: end for
7: for  $i = 1 \rightarrow m$  do
8:    $(u, v) \leftarrow e_i$ 
9:   if  $find\_set(u) \neq find\_set(v)$  then
10:     $T \leftarrow T \cup e_i$ 
11:     $union(u, v)$ 
12:   end if
13: end for
14: return  $T$ 
```

END PROCEDURE

Đánh giá độ phức tạp: Thuật toán Kruskal hoạt động với thời gian $O(m\log m)$ khi được cài đặt bằng cấu trúc dữ liệu DSU.

§3. Bài toán đường đi ngắn nhất

1 Thuật toán Dijkstra

Thuật toán Dijkstra

Algorithm 8 Thuật toán Dijkstra

Input: Đồ thị G và 2 đỉnh xuất phát s .

Output: Đường đi ngắn nhất từ đỉnh s đến các đỉnh còn lại trong đồ thị.

PROCEDURE dijkstra_algorithm(G, l, s):

```

1: for  $v \in V \setminus \{s\}$  do
2:    $\pi(v) \leftarrow \infty$ 
3:    $pred(v) \leftarrow null$ 
4:    $\pi(s) \leftarrow 0$ 
5: end for
6:  $pq \leftarrow priority\_queue()$ 
7: while  $pq$  is not empty do
8:    $u \leftarrow del\_min(pq)$ 
9:   for  $e = (u, v) \in E$  do
10:    if  $\pi(v) > \pi(u) + l_e$  then
11:       $\pi(v) \leftarrow \pi(u) + l_e$ 
12:       $pred(v) \leftarrow u$ 
13:    end if
14:  end for
15: end while

```

END PROCEDURE

Độ phức tạp của thuật toán Dijkstra tương tự như thuật toán Prim.

2 Thuật toán Bellman-Ford

Thuật toán Bellman-Ford

Đánh giá độ phức tạp của thuật toán: Thuật toán hoạt động trong thời gian $O(m \times n)$

Khi thực hiện thuật toán Bellman-Ford tới bước thứ n , nếu thuật toán thực hiện bước thứ n thì đồ thị chứa chu trình âm.

Algorithm 9 Thuật Bellman-Ford

Input: Đồ thị G và 2 đỉnh đích t .

Output: Đường đi ngắn nhất từ các đỉnh trong đồ thị đến đỉnh t .

PROCEDURE bellmanford_algorithm(G, c, t):

```

1: for  $v \in V$  do
2:    $d(v) \leftarrow \infty$ 
3:    $\text{succ}(v) \leftarrow \text{null}$ 
4: end for
5:  $d(t) \leftarrow 0$ 
6: for  $i = 1 \rightarrow n - 1$  do
7:   for  $w \in V$  do
8:     if  $d(w)$  was updated in previous iteration then
9:       for  $(v, w) \in E$  do
10:        if  $d(v) > d(w) + c_{vw}$  then
11:           $d(v) \leftarrow d(w) + c_{vw}$ 
12:           $\text{succ}(v) = w$ 
13:        end if
14:      end for
15:    end if
16:  end for
17:  if no  $d(w)$  changed in current iteration then
18:    return
19:  end if
20: end for

```

END PROCEDURE

§4. Bài toán luồng cực đại, lát cắt cực tiểu

1 Lát cắt và luồng trên mạng

Một luồng trên mạng là đồ thị G với tập đỉnh và tập cạnh (V, E) và 2 đỉnh $s, t \in V$ sao cho s là nút phát và t là nút thu. Trên mỗi cạnh có một trọng số $c(e)$ được gọi là dung tích của cạnh đó.

Một luồng $s - t$ là một ánh xạ f thỏa mãn:

- $0 \leq f(e) \leq c(e), \forall e \in E$
- $\sum_{e \text{ in to } v} f(e) = \sum_{e \text{ out of } v} f(e), \forall v \in V \setminus \{s, t\}$

Giá trị của luồng f được xác định bởi công thức:

$$val(f) = \sum_{e \text{ in to } s} f(e) - \sum_{e \text{ out of } s} f(e)$$

Bài toán luồng cực đại là bài toán xác định luồng f trên đồ thị G có giá trị lớn nhất.

Một lát cắt $s - t$ là một cách phân hoạch tập đỉnh V thành 2 tập A và B sao cho $s \in A$ và $t \in B$. Dung tích của lát cắt (A, B) được ký hiệu và xác định bởi công thức:

$$cap(A, B) = \sum_{e \text{ in to } A} c(e)$$

Bài toán lát cắt cực tiểu là xác định lát cắt (A, B) có dung tích nhỏ nhất.

Mệnh đề: Với (A, B) là lát cắt bất kỳ và f là 1 luồng bất kỳ trên đồ thị G . Khi đó:

$$val(f) = \sum_{e \text{ in to } A} f(e) - \sum_{e \text{ out of } A} f(e)$$

Định lý đối ngẫu yếu: Với (A, B) là lát cắt bất kỳ và f là 1 luồng bất kỳ trên đồ thị G thì:

$$val(f) \leq cap(A, B)$$

Hệ quả: Nếu f là luồng cực đại và (A, B) là lát cắt cực tiểu thì:

$$val(f) = cap(A, B)$$

Định lý luồng cực đại, lát cắt cực tiểu: Giá trị của luồng cực đại bằng dung tích của lát cắt cực tiểu.

2 Thuật toán Ford-Fulkerson

Thuật toán Ford-Fulkerson

Algorithm 10 Thuật toán Ford-Fulkerson

Input: Đồ thị G , nguồn phát s và nguồn thu t .

Output: Luồng cực đại trên G .

PROCEDURE ford_fulkerson(G):

```

1: for  $e \in E_G$  do
2:    $f(e) \leftarrow 0$ 
3: end for
4:  $G_f \leftarrow$  residual network of  $G$  with respect to  $f$ .
5: while (there exists an  $s \rightarrow t$  path  $P$  in  $G_f$ ) do
6:    $f \leftarrow$  augment( $f, c, p$ )
7:   update( $G_f$ )
8: end while
9: return  $f$ 

```

END PROCEDURE

PROCEDURE augment(f, c, p):

```

1:  $b \leftarrow$  bottleneck capacity of path  $P$ 
2: for edge  $e \in P$  do
3:   if  $e \in E$  then
4:      $f(e) \leftarrow f(e) + b$ 
5:   else
6:      $f[e_{reverse}] \leftarrow f[e_{reverse}] - b$ 
7:   end if
8: end for
9: return  $f$ 

```

END PROCEDURE

Để tìm lát cắt cực tiểu (A, B) ta thực hiện thuật toán Ford-Fulkerson để tìm luồng cực đại f . Khi thuật toán kết thúc, với A gồm các đỉnh mà tồn tại đường đi đến nó từ s trên đồ thị mạng còn dư, các đỉnh còn lại thuộc tập B thì (A, B) là lát cắt cực tiểu trên G .

§5. Bài toán cặp ghép

1 Bài toán ghép cặp

Cho một đồ thị vô hướng $G = (V, E)$ với trọng số cạnh c . Một cặp ghép (matching) trong G là một tập con của E gồm các cạnh đôi một không kề nhau. Trọng số của một cặp ghép là tổng trọng số của tất cả các cạnh trong cặp ghép đó.

Bài toán cặp ghép tổng quát là bài toán tìm cặp ghép có trọng số cực tiểu. Bài toán ghép cặp với lực lượng cực đại là bài toán tìm cặp ghép có nhiều cạnh nhất có thể.

Bài toán ghép cặp hoàn hảo (perfect matching) là bài toán tìm cặp ghép thỏa mãn điều kiện mỗi đỉnh phải kề với một cạnh trong cặp ghép, tức là cặp ghép phải có $\frac{|V|}{2}$ cạnh.

Ta có thể biến đổi bài toán tìm cặp ghép hoàn hảo có trọng số cực tiểu về bài ghép cặp (một phần) có trọng số cực tiểu bằng cách giảm trọng số các cạnh một số M đi $1 + \sum_e |c(e)|$. Sau đó tìm cặp ghép có trọng số cực tiểu trên đồ thị mới.

Chứng minh

Gọi $G = (V, E)$ là đồ thị ban đầu. Giả sử $|V| = 2n, n \in \mathbb{N}^*$ và tồn tại một cặp ghép hoàn hảo có trọng số cực tiểu trên G .

Ta xây dựng đồ thị $G' = (V', E')$ sao cho $V' = V, E' = E$ và trọng số của mỗi cạnh $e' \in E'$ được xác định như sau:

$$c_{e'} = c_e - c', \forall e \in E$$

Với $c' = 1 + \sum_{e \in E} |c_e|$. Khi đó:

$$c_{e'} \leq -1, \forall e' \in E'$$

Giả sử M' là 1 cặp ghép có trọng số cực tiểu trên G' . Khi đó M' sẽ chứa $\frac{|V|}{2}$ cạnh vì ngược lại nếu M' không chứa $\frac{|V|}{2}$ cạnh. Ta có thể thêm một số cạnh $\bar{e}$ vào M' sao cho M' vẫn là một cặp ghép (Do trên G tồn tại cặp ghép hoàn hảo nên ta có thể thu được cặp ghép có tối đa $\frac{|V|}{2}$ cạnh trên G'). Do $c_{e'} \leq -1, \forall e' \in E'$ nên

trọng số của cặp ghép sẽ giảm đi và do đó M' không còn là cặp ghép có trọng số cực tiểu (Vô lý).

Xét tập $\mathcal{M}$ gồm toàn bộ các cặp ghép có $\frac{|V|}{2}$ cạnh trong G' . Do M' là cặp ghép có trọng số cực tiểu nên:

$$C_{M'} \leq C_M, \forall M \in \mathcal{M}$$

$$\Rightarrow C_{M'} + nc' \leq C_M + nc', \forall M \in \mathcal{M}$$

Trong đó ký hiệu $C(M)$ là trọng số của cặp ghép M .

Với $\overline{M}$ là một cặp ghép thu được trên G bằng việc sử dụng tập cạnh ghép của M' . Khi đó $\overline{M}$ là một cặp ghép hoàn hảo, hơn nữa trong toàn bộ các cặp ghép hoàn hảo trên G , ký hiệu là tập $\mathcal{M}'$:

$$C_{\overline{M}} = C_{M'} + nc' \leq C_M + nc' = C_M, \forall M \in \mathcal{M}'$$

Do đó $\overline{M}$ là một cặp ghép hoàn hảo có trọng số cực tiểu trên G . □

Ngược lại ta cũng có thể đưa bài toán ghép cặp có trọng số cực tiểu về bài toán ghép cặp hoàn hảo có trọng số cực tiểu bằng cách thêm $|V|$ đỉnh giả để tạo thành tập đỉnh mới V' và xây dựng đồ thị G' trên V' với trọng số các cạnh như sau:

- $c'(e) = c(e)$ nếu $e \in E$
- $c'(e) = 0$ nếu $e \notin E$

Một cặp ghép hoàn hảo có trọng số cực tiểu trên G' là một cặp ghép có trọng số cực tiểu trên G sau khi bỏ đi các cạnh thêm vào.

Bài toán phân công là bài toán ghép cặp hoàn hảo với trọng số cực tiểu trên đồ thị hai phía $G(X \cup Y, E)$ sao cho $|X| = |Y|$.

2 Thuật toán Hungarian

Ví dụ: Một phân xưởng lắp ráp có 4 công nhân được giao 4 công việc. Bảng sau cung cấp số liệu về chi phí cần trả cho mỗi công nhân khi phân cho họ mỗi công việc đó. Bạn hãy giúp quản đốc đưa ra một cách phân việc tốn ít chi phí nhất.

	Job 1	Job 2	Job 3	Job 4
A_1	10	10	12	5
A_2	11	7	3	6
A_3	5	13	17	10
A_4	5	9	12	2

Xét ma trận:

$$A = \begin{pmatrix} 10 & 10 & 12 & 5 \\ 11 & 7 & 3 & 6 \\ 5 & 13 & 17 & 10 \\ 5 & 9 & 12 & 2 \end{pmatrix} \xrightarrow{\substack{H_1 - 5 \\ H_2 - 3 \\ H_3 - 5 \\ H_4 - 2}} \begin{pmatrix} 5 & 5 & 7 & 0 \\ 8 & 4 & 0 & 3 \\ 0 & 8 & 12 & 5 \\ 3 & 7 & 10 & 0 \end{pmatrix} \xrightarrow{C_2 - 4} \begin{pmatrix} 5 & 1 & 7 & 0 \\ 8 & 0 & 0 & 3 \\ 0 & 4 & 12 & 5 \\ 3 & 3 & 10 & 0 \end{pmatrix}$$

Do số phủ tối thiểu là $3 < 4$ nên ta trừ các phần tử chưa bị phủ đi $\min = 1$ và cộng vào các phần tử được phủ bởi cả hàng và cột thêm 1:

$$\begin{pmatrix} 4 & 0 & 6 & 0 \\ 8 & 0 & 0 & 4 \\ 0 & 4 & 12 & 6 \\ 2 & 2 & 9 & 0 \end{pmatrix}$$

Do số phủ tối thiểu là 4 nên ta thu được phương án tối ưu:

$$\begin{pmatrix} 4 & \mathbf{0} & 6 & 0 \\ 8 & 0 & \mathbf{0} & 4 \\ \mathbf{0} & 4 & 12 & 6 \\ 2 & 2 & 9 & \mathbf{0} \end{pmatrix}$$

Cách phân công công việc là:

$$A_1 \rightarrow \text{Job 2}$$

$$A_2 \rightarrow \text{Job 3}$$

$$A_3 \rightarrow \text{Job 1}$$

$$A_4 \rightarrow \text{Job 4}$$

Tổng chi phí thực hiện là: $10 + 3 + 5 + 2 = 20$

§6. Bài toán ghép cặp bền vững

1 Bài toán ghép cặp bền vững

Cho tập n bệnh viện H và n sinh viên S . Mỗi bệnh viện và mỗi sinh viên đều lập cho mình 1 danh sách ưu tiên các sinh viên và bệnh viện tương ứng. Một cách ghép cặp M là tập các cặp có thứ tự $h - s$ sao cho $h \in H$ và $s \in S$ và mỗi bệnh viện h và mỗi sinh viên s xuất hiện trong nhiều nhất trong một cặp $h - s$ trong M . Cách ghép cặp M được gọi là hoàn hảo nếu $|M| = |H| = |S|$.

Cho ghép cặp M , bệnh viện h và sinh viên s tạo thành 1 cặp ghép không ổn định nếu h ưu tiên s hơn sinh viên đang được ghép cặp cùng mình hiện tại và s ưu tiên h hơn bệnh viện đang được ghép cặp cùng mình hiện tại. Một ghép cặp ổn định là một cách ghép cặp mà trong đó không có cặp ghép không ổn định.

Bài toán ghép cặp bền vững là bài toán tìm ra ghép cặp M ổn định (nếu tồn tại).

2 Thuật toán Gale-Shaley

Thuật toán Gale-Shaley

Algorithm 11 Thuật toán Gale-Shaley

Input: Tập bệnh viện H và tập sinh viên S và danh sách ưu tiên P .

Output: Một cách ghép cặp bền vững M .

PROCEDURE gale_shaley(H, S, P):

```

1:  $M \leftarrow \emptyset$ 
2: while (some hospital  $h$  is unmatched and hasn't proposed to every student) do
3:    $s \leftarrow$  first student on  $h$ 's list to whom  $h$  has not yet proposed.
4:   if ( $s$  is unmatched) then
5:      $M \leftarrow M \cup \{h - s\}$ 
6:   else if ( $s$  prefers  $h$  to current partner  $h'$ ) then
7:     Replace  $h' - s$  with  $h - s$  in matching  $M$ 
8:   else
9:      $s$  rejects  $h$ 
10:  end if
11: end while
12: return  $M$ 
```

END PROCEDURE

Ví dụ: Sử dụng thuật toán Gale-Shapley để tìm một trạng thái hôn nhân bền

vững cho 4 đàn ông và 4 phụ nữ có danh sách thứ tự ưu tiên như sau:

Man	Preference list	Woman	Preference list
Adam	Eve, Greta, Hannah, Faith	Eve	Ben, Caleb, David, Adam
Ben	Eve, Faith, Greta, Hannah	Faith	Adam, Ben, Caleb, David
Caleb	Greta, Eve, Faith, Hannah	Greta	Caleb, David, Adam, Ben
David	Faith, Eve, Hannah, Greta	Hannah	Ben, David, Adam, Caleb

Mỗi người đàn ông và mỗi người phụ nữ được ký hiệu bởi chữ cái đầu tiên trong tên của họ: $Man = \{A, B, C, D\}$; $Woman = \{E, F, G, H\}$.

Ta thực hiện thuật toán Gale-Shaley với người đề xuất là những người đàn ông.

Bước	h	s	$Agree/Reject$	M
0	-	-	-	$\emptyset$
1	A	E	$Agree$	$\{AE\}$
2	B	F	$Agree$	$\{BE\}$
3	C	G	$Agree$	$\{BE, CG\}$
4	D	F	$Agree$	$\{BE, CG, DF\}$
5	A	G	$Reject$	$\{BE, CG, DF\}$
6	A	H	$Agree$	$\{BE, CG, DF, AH\}$

Vậy một trạng thái hôn nhân bền vững là:

Adam - Hannah, Ben - Eve, Caleb - Greta, David - Faith

§7. Các mô hình tối ưu nguyên

1 Mô hình IP cho bài toán đường đi ngắn nhất với trọng số cạnh dương.

Cho đồ thị $G = (V, E)$ với trọng số cạnh dương, tìm đường đi ngắn nhất từ đỉnh s đến đỉnh t .

(1) Thiết lập biến:

$$x_e = \begin{cases} 1 & \text{nếu chọn cạnh } e \text{ trong đường đi} \\ 0 & \text{nếu không chọn cạnh } e \text{ trong đường đi} \end{cases}, \forall e \in E$$

(2) Thiết lập hàm mục tiêu:

$$\min \sum_{e \in E} c_e x_e$$

(3) Thiết lập các ràng buộc:

Với ký hiệu $\delta^+(v)$ là tập các cạnh đi ra khỏi v và $\delta^-(v)$ là tập các cạnh đi vào v .

+ Chỉ có 1 cạnh đi ra từ đỉnh s :

$$\sum_{e \in \delta^+(s)} x_e = 1$$

+ Chỉ có 1 cạnh vào đỉnh t :

$$\sum_{e \in \delta^-(t)} x_e = 1$$

+ Các đỉnh ngoài đỉnh s và đỉnh t có số cạnh đi vào bằng số cạnh đi ra (đường đi không dừng lại ở các đỉnh này):

$$\sum_{e \in \delta^+(u)} x_e = \sum_{e \in \delta^-(u)} x_e, \forall u \in V \setminus \{s, t\}$$

2 Mô hình LP cho bài toán luồng cực đại.

Cho luồng trên mạng $G = (V, E, c, s, t)$, tìm luồng cực đại trên mạng.

(1) Thiết lập biến:

$$x_e \in \mathbb{R}^+ \text{ là giá trị luồng gửi đi trên cạnh } e, \forall e \in E$$

(2) Thiết lập hàm mục tiêu:

$$\max \sum_{e \in \delta^+(s)} x_e - \sum_{e \in \delta^-(s)} x_e$$

(3) Thiết lập các ràng buộc:

+ Không vượt quá dung tích:

$$x_e \leq c_e, \forall e \in E$$

+ Bảo toàn luồng:

$$\sum_{x_e \in \delta^+(u)} x_e = \sum_{x_e \in \delta^-(u)} x_e, \forall u \in V \setminus \{s, t\}$$

3 Mô hình IP cho bài toán cây bao trùm nhỏ nhất.

(1) Thiết lập biến:

$$x_e = \begin{cases} 1 & \text{nếu chọn cạnh } e \text{ trong cây} \\ 0 & \text{nếu không chọn cạnh } e \text{ cây} \end{cases}, \forall e \in E$$

(2) Thiết lập hàm mục tiêu:

$$\min \sum_{e \in E} c_e x_e$$

(3) Thiết lập các ràng buộc:

+ Chứa đúng $|V| - 1$ cạnh:

$$\sum_{e \in E} x_e = |V| - 1$$

+ Không có chu trình:

$$\sum_{u, v \in S} x_{uv} \leq |S| - 1, \forall S \subset V, 2 \leq |S| \leq |V| - 1$$

4 Mô hình IP cho bài toán ghép cặp hoàn hảo với trọng số cực tiểu.

(1) Thiết lập biến:

$$x_e = \begin{cases} 1 & \text{nếu chọn cạnh } e \text{ trong cặp ghép} \\ 0 & \text{nếu không chọn cạnh } e \text{ cặp ghép} \end{cases}, \forall e \in E$$

(2) Thiết lập hàm mục tiêu:

$$\min \sum_{e \in E} c_e x_e$$

(3) Thiết lập các ràng buộc:

+ Mỗi đỉnh kề với đúng 1 cạnh trong tập cạnh ghép:

$$\sum_{e \text{ kề với } u} x_e = 1, \forall u \in V$$

5 Mô hình IP cho bài toán phân công

(1) Thiết lập biến:

$$x_{ij} = \begin{cases} 1 & \text{nếu việc } j \text{ được phân công cho người } i \\ 0 & \text{nếu việc } j \text{ không được phân công cho người } i \end{cases}, \forall i, j = 1, 2, \dots, n$$

(2) Thiết lập hàm mục tiêu:

$$\min \sum_{i,j=1}^n c_{ij} x_{ij}$$

(3) Thiết lập các ràng buộc:

+ Mỗi việc được giao cho đúng 1 người:

$$\sum_{i=1}^n x_{ij} = 1, \forall j = 1, 2, \dots, n$$

+ Mỗi người nhận đúng 1 việc:

$$\sum_{j=1}^n x_{ij} = 1, \forall i = 1, 2, \dots, n$$

6 Mô hình IP cho bài toán TSP

Bài toán TSP: Cho n thành phố, giữa 2 thành phố i và j có 1 đường đi với chi phí c_{ij} . Tìm đường đi đi qua tất cả thành phố, mỗi thành phố đi qua đúng 1 lần sao cho tổng chi phí quãng đường là nhỏ nhất.

Gán cho mỗi thành phố một số từ 1 đến n :

(1) Thiết lập biến:

$$x_{ij} = \begin{cases} 1 & \text{nếu cạnh } (i,j) \text{ nằm trong đường đi} \\ 0 & \text{nếu cạnh } (i,j) \text{ không nằm trong đường đi} \end{cases}, \forall i, j = 1, 2, \dots, n, i \neq j$$

(2) Thiết lập hàm mục tiêu:

$$\min \sum_{i,j=1, i \neq j}^n c_{ij}x_{ij}$$

(3) Thiết lập các ràng buộc:

+ Mỗi thành phố đi qua đúng 1 lần:

$$\sum_{i=1, i \neq j}^n x_{ij} = 1, \forall j = 1, 2, \dots, n$$

$$\sum_{j=1, j \neq i}^n x_{ij} = 1, \forall i = 1, 2, \dots, n$$

+ Chỉ có 1 đường đi đơn đi qua tất cả các đỉnh:

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1, \forall S \subset V, 2 \leq |S| \leq |V| - 1$$

7 Mô hình IP cho bài toán Knapsack

Bài toán Knapsack: Cho n đồ vật, mỗi đồ vật có cân nặng là w và giá trị v , một balo có thể đựng được tối đa W cân nặng. Chọn các đồ vật để đưa vào balo sao cho đạt giá trị lớn nhất.

Gán cho mỗi thành phần một số từ 1 đến n :

(1) Thiết lập biến:

$$x_i = \begin{cases} 1 & \text{nếu chọn đồ vật thứ } i \\ 0 & \text{nếu không chọn đồ vật thứ } i \end{cases}, \forall i = 1, 2, \dots, n$$

(2) Thiết lập hàm mục tiêu:

$$\max \sum_{i=1}^n v_i x_i$$

(3) Thiết lập các ràng buộc:

+ Tổng cân nặng của các đồ vật được chọn không vượt quá W .

$$\sum_{i=1}^n v_i w_i \leq W$$